การประชุมวิชาการระดับชาติด้านวิทยาศาสตร์และเทคโนโลยี ครั้งที่ 10
วันที่ 25-26 กุมภาพันธ์ 2569 ISBN: xxx – xxx – xxx – xxx – x

การประยุกต์ระบบประมวลผลแบบกลุ่มเมฆสำหรับแอปพลิเคชัน Keep Going ด้วย Supabase

CLOUD COMPUTING APPLICATION FOR KEEP GOING USING SUPABASE

[ชื่อ–สกุลผู้จัดทำ ภาษาไทย]
[AUTHOR NAME IN ENGLISH]
[สาขาวิชา/คณะ/มหาวิทยาลัย]
*อีเมลผู้รับผิดชอบบทความ: [กรอกอีเมล]

บทคัดย่อ

โครงงานนี้มีวัตถุประสงค์เพื่อประยุกต์บริการประมวลผลแบบกลุ่มเมฆสำหรับจัดเก็บ ซิงก์ และควบคุมสิทธิ์ข้อมูลของแอปพลิเคชัน Keep Going ระบบใช้ Supabase Auth สำหรับพิสูจน์ตัวตน PostgreSQL สำหรับข้อมูลเชิงโครงสร้าง และ private Storage สำหรับไฟล์เสียง การออกแบบฐานข้อมูลแยกข้อมูลเป็นตารางการตั้งค่าผู้ใช้ เช็กอินรายวัน ไดอารี่ และข้อความที่ไม่ได้ส่ง พร้อมกำหนดความสัมพันธ์กับบัญชีผู้ใช้ ทุกตารางเปิดใช้ Row Level Security และมีนโยบายสำหรับการอ่าน เพิ่ม แก้ไข และลบ โดยตรวจสอบเจ้าของข้อมูลผ่านรหัสผู้ใช้จากโทเคน ระบบฝั่งแอปใช้ publishable key และเก็บ session ใน AsyncStorage ข้อมูลบนคลาวด์เป็นแหล่งข้อมูลสำหรับการใช้งานข้ามอุปกรณ์ ขณะที่อุปกรณ์เก็บสำเนาไว้รองรับการเชื่อมต่อที่ไม่ต่อเนื่อง ผลการพัฒนาพบว่า migration ถูกนำขึ้นฐานข้อมูลระยะไกลครบถ้วน โค้ดผ่านการตรวจชนิดข้อมูล และ Supabase database lint ไม่พบข้อผิดพลาดของ schema อย่างไรก็ตาม ยังต้องทดสอบการชนกันของข้อมูลและกลยุทธ์ซิงก์แบบออฟไลน์ภายใต้เครือข่ายจริงในงานต่อไป

คำสำคัญ: การประมวลผลแบบกลุ่มเมฆ, Supabase, PostgreSQL, การซิงก์ข้อมูล, ความมั่นคงปลอดภัย

ABSTRACT

This project applied cloud computing services to the storage, synchronization, and authorization of data generated by the Keep Going application. Supabase Auth provides user authentication, PostgreSQL stores structured records, and a private Storage bucket holds voice recordings. The database separates user settings, daily check-ins, diary entries, and unsent messages into related tables. Row Level Security is enabled on every exposed table, with select, insert, update, and delete policies that compare each record owner with the authenticated user identifier. The mobile client uses a publishable key and persists its session in AsyncStorage. Cloud data supports account-based retrieval across devices, while a local copy preserves recent state during intermittent connectivity. Both database migrations were applied to the linked remote project, the application passed TypeScript checking, and Supabase database lint reported no schema errors. Further work should evaluate conflict resolution, retry queues, and offline synchronization under real network conditions.

Keywords: Cloud computing, Supabase, PostgreSQL, Data synchronization, Security

1. บทนำ

แอปพลิเคชันที่บันทึกอารมณ์ ไดอารี่ และเสียงสร้างข้อมูลที่มีความเป็นส่วนตัวสูง หากจัดเก็บเฉพาะในอุปกรณ์ข้อมูลอาจสูญหายเมื่อถอนการติดตั้งหรือเปลี่ยนโทรศัพท์ แต่การส่งข้อมูลขึ้นบริการคลาวด์โดยไม่มีการควบคุมสิทธิ์ที่เหมาะสมก็เพิ่มความเสี่ยงต่อการเข้าถึงข้อมูลของผู้อื่น โครงงานนี้จึงประยุกต์บริการคลาวด์เพื่อรองรับบัญชีผู้ใช้ การจัดเก็บแบบมีโครงสร้าง ไฟล์ส่วนตัว และการซิงก์ข้อมูล

วัตถุประสงค์คือ 1) ออกแบบฐานข้อมูลที่รองรับข้อมูลหลักของ Keep Going 2) เชื่อมระบบพิสูจน์ตัวตนเข้ากับแอปมือถือ 3) ใช้ Row Level Security จำกัดข้อมูลตามเจ้าของ 4) จัดเก็บไฟล์เสียงในพื้นที่ส่วนตัว และ 5) รักษาสำเนาในอุปกรณ์เพื่อรองรับการเชื่อมต่อที่ไม่ต่อเนื่อง

2. เอกสารและงานวิจัยที่เกี่ยวข้อง

Supabase เป็นแพลตฟอร์ม Backend as a Service ที่รวมบริการ Auth, PostgreSQL, Data API และ Storage ไว้ในโครงการเดียว สถาปัตยกรรม Auth ประกอบด้วย client SDK, API gateway, บริการ Auth และฐานข้อมูล PostgreSQL โดย client SDK ช่วยจัดการการเรียก API การเก็บ session และการต่ออายุโทเคน (Supabase, 2026a)

สำหรับแอปฝั่งไคลเอนต์ Supabase แนะนำให้ใช้ publishable key ร่วมกับ Row Level Security และสิทธิ์ขั้นต่ำที่จำเป็น โดยห้ามฝัง secret key หรือ service role key ในแอป เนื่องจากกุญแจดังกล่าวสามารถข้าม RLS ได้ (Supabase, 2026b) แนวทางนี้สอดคล้องกับระบบที่พัฒนา ซึ่งใช้โทเคนของผู้ใช้และ auth.uid() เพื่อกำหนดเงื่อนไขเจ้าของแถวข้อมูล

Supabase Storage ใช้นโยบายบนตาราง storage.objects เพื่อควบคุมการอ่านและเขียนไฟล์ โครงงานกำหนดโฟลเดอร์ระดับแรกเป็นรหัสผู้ใช้ และสร้าง signed URL อายุจำกัดเมื่อต้องเล่นไฟล์เสียง ทำให้ bucket สามารถคงสถานะ private และไม่ต้องเปิดไฟล์ต่อสาธารณะ (Supabase, 2026c)

3. วิธีการวิจัย

งานพัฒนาเริ่มจากการวิเคราะห์โมเดลข้อมูลในแอป แล้วแยกข้อมูลตามรูปแบบการใช้งานและวงจรชีวิต ตาราง user_settings มีหนึ่งแถวต่อบัญชี ตาราง daily_checkins จำกัดหนึ่งแถวต่อวันและผู้ใช้ ส่วน diary_entries และ urge_entries รองรับหลายรายการ จากนั้นสร้าง migration ด้วย Supabase CLI และนำขึ้นโครงการระยะไกล

ตารางที่ 1 โครงสร้างข้อมูลบนระบบคลาวด์

ตาราง/พื้นที่จัดเก็บ	ข้อมูลสำคัญ	ข้อกำหนด
auth.users	บัญชี อีเมล และข้อมูลกำกับผู้ใช้	จัดการโดย Supabase Auth
user_settings	วันเริ่มต้น เป้าหมาย onboarding เหตุผล และการแจ้งเตือน	หนึ่งแถวต่อผู้ใช้
daily_checkins	วันที่ อารมณ์ emoji และข้อความ	ไม่ซ้ำตาม user_id และ checkin_date
diary_entries	หัวข้อ เนื้อหา อารมณ์ และเวลาสร้าง	หลายรายการต่อผู้ใช้
urge_entries	ข้อความ จดหมาย ผู้รับ เสียง และระยะเวลา	หลายรายการต่อผู้ใช้
voice-notes	ไฟล์เสียง .m4a	private bucket แยกโฟลเดอร์ตาม user_id

ทุกตารางอ้างอิง auth.users ด้วย foreign key และกำหนด on delete cascade เพื่อไม่ให้มีข้อมูลกำพร้า ตารางเปิด RLS และสร้าง policy แยกตามคำสั่ง select, insert, update และ delete เงื่อนไขใช้ auth.uid() เทียบกับ user_id ทั้งใน using และ with check เพื่อป้องกันการอ่านข้อมูลของผู้อื่นและการเปลี่ยนเจ้าของรายการระหว่างอัปเดต

ตารางที่ 2 การควบคุมสิทธิ์ข้อมูล

การดำเนินการ	เงื่อนไข RLS	ผลที่ต้องการ
SELECT	auth.uid() = user_id	อ่านได้เฉพาะรายการของบัญชีปัจจุบัน
INSERT	WITH CHECK auth.uid() = user_id	สร้างรายการในชื่อของตนเองเท่านั้น
UPDATE	USING และ WITH CHECK ตรงกับ user_id	แก้ไขของตนเองและห้ามย้ายเจ้าของ
DELETE	auth.uid() = user_id	ลบได้เฉพาะรายการของตนเอง
STORAGE	bucket ตรงกันและโฟลเดอร์แรกเป็น user_id	แยกไฟล์เสียงระหว่างบัญชี

ฝั่งแอปสร้าง Supabase client จาก Project URL และ publishable key ที่อ่านจากตัวแปรสภาพแวดล้อม Session ถูกเก็บใน AsyncStorage และเปิดการต่ออายุโทเคนเมื่อแอปอยู่ foreground การบันทึกข้อมูลใช้ upsert สำหรับข้อมูลที่มีข้อกำหนดไม่ซ้ำ และใช้ signed URL อายุจำกัดสำหรับเปิดไฟล์เสียงจาก private bucket

กระบวนการซิงก์เริ่มจากโหลดสำเนาในเครื่องและข้อมูลจากคลาวด์ หากคลาวด์ยังว่าง ระบบย้ายข้อมูลเดิมในเครื่องขึ้นไป หากพบข้อมูลทั้งสองฝั่ง ระบบรวมรายการตามวันที่หรือรหัสรายการ โดยให้ข้อมูลคลาวด์เป็นค่าหลัก แล้ว upsert ข้อมูลที่ยังขาดกลับขึ้นระบบ การล้างข้อมูลเรียกคำสั่งลบทั้งตารางและไฟล์ใน Storage ก่อนลบสำเนาในเครื่อง

4. ผลการวิจัย

ผลการพัฒนาได้ระบบคลาวด์ที่รองรับข้อมูลทุกกลุ่มของแอป Keep Going และแยกความรับผิดชอบเป็นบริการพิสูจน์ตัวตน ฐานข้อมูล และพื้นที่ไฟล์ Migration จำนวนสองรายการถูกนำขึ้นโครงการ Supabase ที่เชื่อมไว้สำเร็จ รายการแรกสร้าง daily_checkins ส่วนรายการที่สองสร้าง user_settings, diary_entries, urge_entries และ private bucket สำหรับเสียง

ตารางที่ 3 ผลการตรวจสอบระบบคลาวด์

รายการตรวจสอบ	ผล
สถานะ migration ระหว่าง local และ remote	ตรงกันทั้ง 2 รายการ
TypeScript type checking	ผ่านโดยไม่พบข้อผิดพลาด
Supabase database lint ระดับ warning	ไม่พบ schema error
การกำหนด RLS	เปิดในทุกตารางข้อมูลแอปและกำหนด policy ครบ CRUD
ความลับฝั่งไคลเอนต์	ใช้เฉพาะ publishable key; ไม่เก็บ database password หรือ secret key ในไฟล์ commit

ผลดังกล่าวยืนยันความพร้อมด้านโครงสร้างและการเชื่อมต่อของระบบ แต่ยังไม่ใช่ผลทดสอบสมรรถนะภายใต้ปริมาณผู้ใช้สูง การทำงานข้ามอุปกรณ์พร้อมกัน หรือการกู้คืนจากความล้มเหลวของเครือข่าย

4.1 โค้ดที่ใช้เชื่อมต่อและจัดการฐานข้อมูล

src/services/supabase.ts

Source lines 1–19

```
1   import AsyncStorage from '@react-native-async-storage/async-storage';
2   import { createClient } from '@supabase/supabase-js';
3   import 'react-native-url-polyfill/auto';
4
5   const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL;
6   const supabasePublishableKey =
↳   process.env.EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY;
7
8   if (!supabaseUrl || !supabasePublishableKey) {
9     throw new Error('Supabase environment variables are missing');
10  }
11
12  export const supabase = createClient(supabaseUrl, supabasePublishableKey, {
13    auth: {
14      storage: AsyncStorage,
15      autoRefreshToken: true,
16      persistSession: true,
17      detectSessionInUrl: false,
18    },
19  });
```

ภาพที่ 1 การเชื่อมต่อ Supabase

ที่มา src/services/supabase.ts บรรทัด 1–19

บรรทัด 5–6 อ่าน Project URL และ publishable key จากตัวแปรสภาพแวดล้อม บรรทัด 8–10 ตรวจว่ากำหนดค่าครบก่อนสร้าง client ภาพแสดงเฉพาะชื่อตัวแปร โดยไม่มีค่ากุญแจหรือรหัสผ่าน

บรรทัด 12 สร้าง supabase client ที่บริการข้อมูลนำไปใช้ร่วมกัน บรรทัด 14–17 กำหนดให้ AsyncStorage เก็บ session ต่ออายุโทเคนอัตโนมัติ และคงการเข้าสู่ระบบเมื่อเปิดแอปอีกครั้ง ส่วนนี้เป็นการตั้งค่าการเชื่อมต่อและ Auth ยังไม่ใช่คำสั่งเพิ่มแถวในฐานข้อมูล

4.1 โค้ดที่ใช้เชื่อมต่อและจัดการฐานข้อมูล ต่อ

supabase/migrations/20261021062416_create_daily_checkins.sql

Source lines 1–30

```
1    create table public.daily_checkins (
2      id uuid primary key default gen_random_uuid(),
3      user_id uuid not null references auth.users(id) on delete cascade,
4      checkin_date date not null,
5      mood text not null,
6      emoji text not null,
7      note text not null default '',
8      created_at timestamptz not null default now(),
9      updated_at timestamptz not null default now(),
10     unique (user_id, checkin_date)
11   );
12
13   create index daily_checkins_user_date_idx
14     on public.daily_checkins (user_id, checkin_date desc);
15
16   alter table public.daily_checkins enable row level security;
17   grant select, insert, update, delete on public.daily_checkins to
↳    authenticated;
18
19   create policy "Users can read their own daily check-ins"
20     on public.daily_checkins for select to authenticated
21     using ((select auth.uid()) = user_id);
22
23   create policy "Users can create their own daily check-ins"
24     on public.daily_checkins for insert to authenticated
25     with check ((select auth.uid()) = user_id);
26
27   create policy "Users can update their own daily check-ins"
28     on public.daily_checkins for update to authenticated
29     using ((select auth.uid()) = user_id)
30     with check ((select auth.uid()) = user_id);
```

ภาพที่ 2 โครงสร้างตารางและสิทธิ์ข้อมูล

ที่มา supabase/migrations/20261021062416_create_daily_checkins.sql บรรทัด 1–30

บรรทัด 1–11 สร้างตาราง daily_checkins โดย id เป็น UUID และ user_id อ้างอิงบัญชีใน auth.users ฟิลด์ checkin_date, mood, emoji และ note เก็บวันที่ อารมณ์ อีโมจิ และข้อความ เงื่อนไข unique (user_id, checkin_date) จำกัดให้แต่ละบัญชีมีหนึ่งแถวต่อวัน

บรรทัด 16 เปิด Row Level Security และบรรทัด 17 ให้สิทธิ์ CRUD แก่ผู้ที่เข้าสู่ระบบ นโยบายในบรรทัด 19–30 ตรวจ auth.uid() เทียบ user_id เพื่อจำกัดการอ่าน เพิ่ม และแก้ไขเฉพาะข้อมูลของตนเอง ทั้ง using และ with check ใช้ตรวจแถวตามการดำเนินการ ส่วน policy การลบอยู่ถัดจากช่วงโค้ดในภาพ

4.1 โค้ดที่ใช้เชื่อมต่อและจัดการฐานข้อมูล ต่อ

src/services/dailyCheckins.ts

Source lines 25–43

```
25  export async function upsertDailyCheckin(entry: Entry): Promise<Entry> {
26    const { data: authData, error: authError } = await supabase.auth.getUser();
27    if (authError || !authData.user) throw authError ?? new
↳   Error('กรุณาเข้าสู่ระบบ');
28
29    const { data, error } = await supabase
30      .from('daily_checkins')
31      .upsert({
32        user_id: authData.user.id,
33        checkin_date: entry.date,
34        mood: entry.mood,
35        emoji: entry.emoji,
36        note: entry.note,
37        updated_at: new Date().toISOString(),
38      }, { onConflict: 'user_id,checkin_date' })
39      .select('id, checkin_date, mood, emoji, note')
40      .single();
41    if (error) throw error;
42    return rowToEntry(data as DailyCheckinRow);
43  }
```

ภาพที่ 3 คำสั่งบันทึกเช็กอินลงฐานข้อมูล

ที่มา src/services/dailyCheckins.ts บรรทัด 25–43

บรรทัด 26–27 ตรวจผู้ใช้ปัจจุบันด้วย supabase.auth.getUser() หากไม่พบบัญชีหรือเกิดข้อผิดพลาดจะหยุดการทำงาน จากนั้นเลือกตาราง daily_checkins และส่ง user_id พร้อมวันที่ อารมณ์ อีโมจิ ข้อความ และเวลาแก้ไข

คำสั่ง upsert ใช้ onConflict: user_id,checkin_date ให้ตรงกับ unique constraint ใน SQL ถ้ามีแถววันเดียวกันของบัญชีนั้นจะอัปเดตแถวเดิม หากยังไม่มีจะเพิ่มใหม่ select และ single ขอข้อมูลแถวที่บันทึกกลับมา ส่วน rowToEntry แปลงชื่อฟิลด์ฐานข้อมูลเป็นรูปแบบที่แอปใช้งาน ฟังก์ชันโยน error ให้ผู้เรียกจัดการเมื่อบันทึกไม่สำเร็จ

4.1 โค้ดที่ใช้เชื่อมต่อและจัดการฐานข้อมูล ต่อ

src/services/cloudData.ts

Source lines 100–118

```
100  export async function upsertDiaryEntry(entry: DiaryEntry) {
101    const userId = await currentUserId();
102    const { error } = await supabase.from('diary_entries').upsert({
103      user_id: userId,
104      id: entry.id,
105      created_at: entry.createdAt,
106      title: entry.title,
107      content: entry.content,
108      mood: entry.mood ?? null,
109      emoji: entry.emoji ?? null,
110    }, { onConflict: 'user_id,id' });
111    if (error) throw error;
112  }
113
114  export async function deleteCloudDiaryEntry(id: string) {
115    const userId = await currentUserId();
116    const { error } = await
↳    supabase.from('diary_entries').delete().eq('user_id', userId).eq('id', id);
117    if (error) throw error;
118  }
```

ภาพที่ 4 การเพิ่มและลบไดอารี

ที่มา src/services/cloudData.ts บรรทัด 100–118

ฟังก์ชัน upsertDiaryEntry อ่านรหัสผู้ใช้แล้วส่งข้อมูลไปตาราง diary_entries โดยจับคู่ id, createdAt, title, content, mood และ emoji กับคอลัมน์ฐานข้อมูล กำหนดอารมณ์และอีโมจิที่ไม่มีค่าเป็น null ตาม schema

onConflict: user_id,id ใช้คู่รหัสบัญชีและรหัสรายการเพื่อเพิ่มหรือแก้ไขรายการเดิม ส่วน deleteCloudDiaryEntry ใช้ delete ร่วมกับ eq สองเงื่อนไข ให้ลบเฉพาะ id ที่ระบุและเป็นของบัญชีปัจจุบัน การกรองจาก client ทำงานร่วมกับ RLS ที่ฐานข้อมูล มิใช่ใช้แทนนโยบายสิทธิ์

4.1 โค้ดที่ใช้เชื่อมต่อและจัดการฐานข้อมูล ต่อ

src/context/AppContext.tsx

Source lines 109–132

```
109    const addEntry = (mood: Mood, note: string) => {
110      const entry: Entry = { id: String(Date.now()), date: isoToday(), mood:
↳    mood.label, emoji: mood.emoji, note };
111      setState((current) => ({ ...current, entries: [entry,
↳    ...current.entries.filter((item) => item.date !== entry.date)] }));
112      if (user) {
113        void upsertDailyCheckin(entry).then((savedEntry) => {
114          setState((current) => ({
115            ...current,
116            entries: current.entries.map((item) => item.date === savedEntry.date
↳     ? savedEntry : item),
117          }));
118        }).catch(() => undefined);
119      }
120    };
121
122    const addDiaryEntry = (title: string, content: string, mood?: Mood) => {
123      const diaryEntry: DiaryEntry = {
124        id: String(Date.now()),
125        createdAt: new Date().toISOString(),
126        title: title.trim() || 'บันทึกของฉัน',
127        content: content.trim(),
128        mood: mood?.label,
129        emoji: mood?.emoji,
130      };
131      setState((current) => ({ ...current, diaryEntries: [diaryEntry,
↳    ...current.diaryEntries] }));
132      if (user) void upsertDiaryEntry(diaryEntry).catch(() => undefined);
```

ภาพที่ 5 จุดเชื่อมจากแอปไปยังบริการฐานข้อมูล

ที่มา src/context/AppContext.tsx บรรทัด 109–132

addEntry สร้างข้อมูลเช็กอินและปรับ state ในแอปก่อน เมื่อมีผู้ใช้ที่เข้าสู่ระบบจึงเรียก upsertDailyCheckin จากภาพที่ 3 หลังเซิร์ฟเวอร์ตอบกลับ ระบบนำ savedEntry มาแทนข้อมูลวันเดียวกัน ทำให้ใช้ id และข้อมูลที่ฐานข้อมูลส่งกลับ

addDiaryEntry เตรียมหัวข้อ เนื้อหา และเวลา เพิ่มรายการเข้า state แล้วเรียก upsertDiaryEntry จากภาพที่ 4 เส้นทางการบันทึกจึงเป็นหน้าจอเรียก Context จากนั้น Context เรียก service และ service ส่งข้อมูลไป Supabase

ข้อจำกัดคือ catch(() => undefined) ในภาพละทิ้งข้อผิดพลาดของการส่งข้อมูล ผู้ใช้จึงอาจเห็นรายการใน state แม้การบันทึกบนคลาวด์ล้มเหลว ภาพนี้ยืนยันจุดเชื่อมในโค้ด แต่ไม่ยืนยันว่าทุกรายการส่งสำเร็จ ควรเพิ่มสถานะ pending, synced และ failed พร้อมการแจ้งเตือนหรือส่งซ้ำ

5. อภิปรายผลและข้อเสนอแนะการวิจัย

การใช้บริการแบบ managed ลดภาระการสร้างเซิร์ฟเวอร์พิสูจน์ตัวตนและ API สำหรับ CRUD ด้วยตนเอง ขณะเดียวกัน PostgreSQL ทำให้กำหนด constraint, foreign key และนโยบาย RLS ใกล้กับข้อมูลได้ การใช้ unique constraint ของ daily_checkins ช่วยบังคับกฎหนึ่งรายการต่อวันในชั้นฐานข้อมูล ไม่ต้องพึ่งการตรวจสอบจากหน้าจอเพียงอย่างเดียว

ข้อจำกัดปัจจุบันคือกลยุทธ์ออฟไลน์ยังเป็นการเก็บสำเนาและซิงก์เมื่อเริ่มใช้งาน มิใช่คิวปฏิบัติการที่รับประกันการส่งซ้ำทุกคำสั่ง หากแก้ข้อมูลเดียวกันจากหลายอุปกรณ์พร้อมกันยังไม่มีการตรวจเวอร์ชันหรือ conflict resolution นอกจากนี้ signed URL ของเสียงมีอายุจำกัดและต้องสร้างใหม่เมื่อหมดอายุ

งานต่อไปควรเพิ่มตารางหรือคิวติดตามสถานะ pending, synced และ failed ใช้ updated_at ร่วมกับ revision หรือ optimistic concurrency control ทดสอบ network interruption และ retry เพิ่มการบันทึกเหตุการณ์ที่ไม่รวมเนื้อหาส่วนตัว และวางนโยบายสำรองข้อมูล ระยะเวลาเก็บรักษา การลบบัญชี และการตอบสนองต่อเหตุการณ์ด้านความมั่นคงปลอดภัยให้ชัดเจน

6. เอกสารอ้างอิง

PostgreSQL Global Development Group. (2026). PostgreSQL row security policies. Retrieved October 21, 2026, from https://www.postgresql.org/docs/current/ddl-rowsecurity.html

Supabase. (2026a). Auth architecture. Retrieved October 21, 2026, from https://supabase.com/docs/guides/auth/architecture

Supabase. (2026b). Securing your data. Retrieved October 21, 2026, from https://supabase.com/docs/guides/database/secure-data

Supabase. (2026c). Storage access control. Retrieved October 21, 2026, from https://supabase.com/docs/guides/storage/security/access-control

Supabase. (2026d). Use Supabase Auth with React Native. Retrieved October 21, 2026, from https://supabase.com/docs/guides/auth/quickstarts/react-native